Problem 1.C **Miss-rate analysis**

Now Ben is studying the effect of set-associativity on the cache performance. Since he now knows the access time of each configuration, he wants to know the miss-rate of each one. For the miss-rate analysis, Ben is considering two small caches: a direct-mapped cache with 8 lines with 32 bytes/line, and a 4-way set-associative cache of the same size and line size. For the set-associative cache, Ben tries out two replacement policies – least recently used (LRU) and round robin (FIFO).

Ben tests the cache by accessing the following sequence of hexadecimal byte addresses, starting with empty caches. For simplicity, assume that the ***addresses are only 12 bits***. **Complete the following tables** for the direct-mapped cache and both types of 4-way set-associative caches showing the progression of cache contents as accesses occur (in the tables, 'inv' = invalid, and the column of a particular cache line contains the tag of that line). Also, for each address calculate the tag and index (which should help in filling out the table). *You only need to fill in elements in the table when a value changes.*

D-map	**Addresses and tags are in HEX**								
	line in cache (tag)								hit?
Address	L0	L1	L2	L3	L4	L5	L6	L7	
11B	1	inv	inv	inv	inv	inv	inv	inv	no
134		1							no
20D	2								no
1A2						1			no
105	1								no
360				3					no
27D				2					no
121		1							yes
1A3						1			yes
17A				1					no
307	3								no
273				2					no
131		1							yes

	Direct-Mapped
Total Misses	10
Total Accesses	13

4-way **Address**	**LRU -- addresses and tags are in HEX**								
	line in cache								hit?
	Set 0				Set 1				
	way0	**way1**	**Way2**	**way3**	**way0**	**way1**	**way2**	**way3**	
11B	4	inv	inv	inv	inv	inv	inv	inv	no
134					4				no
20D		8							no
1A2						6			no
105	X								yes
360							D		no
27D								9	no
121					X				yes
1A3						X			yes
17A							5		no
307			C						no
273								X	yes
131					X				yes

	4-way LRU
Total Misses	8
Total Accesses	13

4-way **Address**	**FIFO -- addresses and tags are in HEX**								
	line in cache (tag)								hit?
	Set 0				Set 1				
	way0	**way1**	**way2**	**way3**	**way0**	**way1**	**way2**	**way3**	
11B	4	inv	inv	inv	inv	inv	inv	inv	no
134					4				no
20D		8							no
1A2						6			no
105	X								yes
360							D		no
27D								9	no
121					X				yes
1A3						X			yes
17A					5				no
307			C						no
273								X	yes
131					4				no

	4-way FIFO
Total Misses	9
Total Accesses	13

Problem 4.A **Baseline Cache Design**

The diagram below shows our victim cache, a 32-Byte fully associative cache with four 8-Byte cache lines. Each line contains of two 4-Byte words and has an associated tag and two status bits (valid and dirty). The Input Address is 32-bits and the two least significant bits are assumed to be zero. The output of the cache is a 32-bit word.

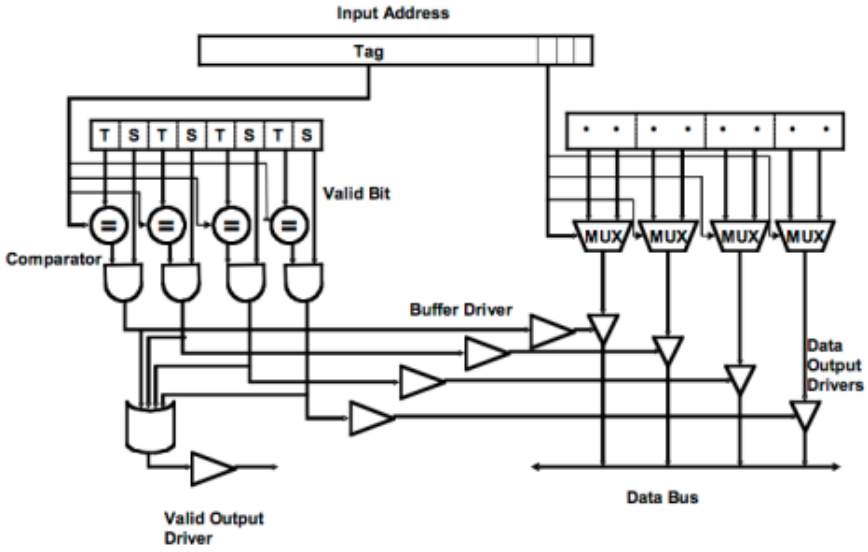


Figure 2.5-1: Victim cache datapath

Please complete Table 2.5-1 with delays across each element of the cache. Using the data you compute in Table 2.5-1, calculate the critical path delay through this cache (from when the Input Address is set to when both Valid Output Driver and the appropriate Data Output Driver are outputting valid data).

Component	Delay equation (ps)	FA(ps)
Comparator	30×(# of tag bits) + 100	970
N-to-1 MUX	$50\times\log_2 N + 100$	150
Buffer driver	200	200
AND gate	100	100
OR gate	$50\times \log_2 N + 100$	150
Data output driver	50×(associativity) + 100	300
Valid output driver	100	100

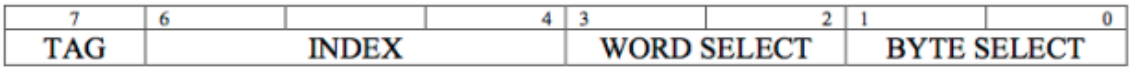
Table 2.5-1: Delay of each cache component

Critical Path Cache Delay:

Problem 4.B **Victim Cache Behavior**

Now we will study the impact of a victim cache on cache hit rate. Our main L1 cache is a 128 byte, direct-mapped cache with 16 bytes per cache line. The cache is word (4-bytes) addressable. The victim cache in Figure 2.5-1 is a 32-byte fully associative cache with 16 bytes per cache line, and is also word addressable. The victim cache uses the first in first out (FIFO) replacement policy.

Please complete Table 2.5-2 showing a trace of memory accesses. In the table, each entry contains the tag of that line, or "inv", if no data is present. You should only fill in elements in the table when a value changes. For simplicity, the addresses are only 8 bits. The first 3 lines of the table have been filled in for you. For your convenience, the address breakdown for access to the main cache is depicted below.



Input Address	Main Cache (tag)									Victim Cache (tag)		
	L0	L1	L2	L3	L4	L5	L6	L7	Hit?	Way0	Way1	Hit?
	inv	inv	inv	inv	inv	inv	inv	inv	-	inv	inv	-
0	0								N			N
80	1								N	0		N
4	0								N	8		Y
A0			1						N			N
10		0							N			N
C0					1				N			N
18		0							Y			
20			0						N		A	N
8C	1								N	0		Y
28			0						Y			
AC			1						N		2	Y
38				0					N			N
C4					1				Y			
3C				0					Y			
48					0				N	C		N
0C	0								N		8	N
24			0						N	A		N

Table 2.5-2: Memory access trace